

Problema do Bar dos Filósofos 2026.1



Problema do “Bar dos Filósofos”, também conhecido como Drinking Philosophers Problem, é uma generalização do problema clássico “Jantar dos Filósofos”, proposto por E.W. Dijkstra em 1971. Em 1984, Chandy e Misra apresentaram uma generalização deste problema, o Bar dos Filósofos, mostrado adiante.

Esse tipo de problema é emblemático na área da computação pois pode representar praticamente todos os problemas de programação concorrente e computação paralela em um único exemplo. O principal objetivo é demonstrar o controle de acesso a recursos compartilhados com exclusão mútua sem ocorrência de deadlock.

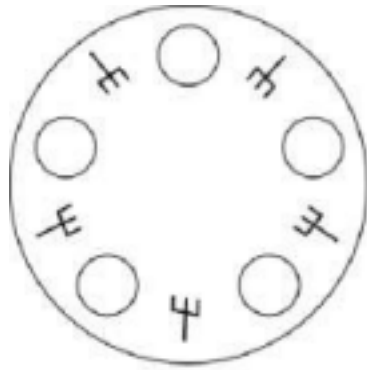
Para este trabalho deverão ser criados grupos com mínimo de 2 e máximo de 4 alunos. Deverão ser implementadas, no mínimo, duas soluções, conforme indicadas abaixo, e comparar o resultado. A ideia é que cada aluno implemente uma solução.

A proposta do trabalho é desenvolver uma implementação conforme descrito a seguir.

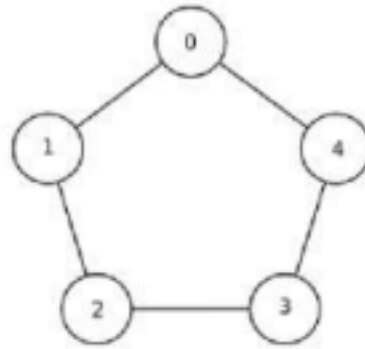
Problema do Jantar dos Filósofos

Antes de descrever o problema do Bar dos Filósofos, vamos descrever o Problema do Jantar dos Filósofos.

Cinco filósofos se reúnem para jantar e conversar em uma mesa redonda com um garfo colocado sobre a mesa entre cada um deles. Como discutir problemas relacionados filosofia é um exaustivo, eles precisam comer de vez em quando.



(a) Problem setting



(b) Formalization as graph

A fim de fazer isso, um filósofo requer o garfo da esquerda e o da direita, que são compartilhados com os colegas da esquerda e da direita, respectivamente. Depois de um filósofo comer, os dois garfos são colocados de volta na mesa, possibilitando que outro filósofo possa pegar os garfos e iniciar o jantar. É importante notar que um filósofo deve possuir ambos os garfos, caso contrário, ele não pode comer nada. Devido ao fato de que há apenas cinco garfos e cada filósofo precisa de dois deles para comer, é óbvio que, no máximo, dois filósofos podem comer simultaneamente. Uma solução para este problema tem de garantir que vários filósofos possam comer ao mesmo tempo. Além disso, deadlocks (impasses) e starvation (fome) devem ser evitados.

A descrição do Problema do Jantar dos Filósofos é bastante restrita. O problema se resume um grafo em ciclo, onde cada processo sempre precisa de recursos ao seu lado para executar sua seção crítica.

Problema do Bar dos Filósofos

A principal diferença nesse problema é o grafo de modelagem, que ao contrário do grafo circular do jantar dos filósofos, agora temos um grafo genérico. Nenhuma restrição é feita para a estrutura do grafo, como distribuição ou grau de conectividade. Os filósofos (processos) são representados pelo nós (vértices) do grafo enquanto as garrafas (recursos compartilhados) são representadas pelas arestas do grafo.

A generalização é focada em dois pontos: (1) a estrutura do grafo é arbitrária, e, (2) o número e conjunto de garrafas requeridas podem variar. Em cada rodada, cada processo escolhe um subconjunto de suas garrafas adjacentes para esta sessão de bebida. Este subconjunto escolhido não tem necessariamente que ser a mesma em

cada sessão de bebida. Se o filósofo escolher todas as garrafas adjacentes em um grafo circular, podemos simular o problema do jantar dos filósofos. Os estados dos filósofos podem ser: tranquilo, com sede e bebendo, semelhante aos estados filosofando, com fome e comendo, no problema do jantar dos filósofos. Cada filósofo pode requerer no mínimo uma e no máximo n garrafas, sendo n o número de arestas associadas a cada vértice.

Descrição da Implementação

O trabalho prático consiste em escrever um programa na linguagem de escolha do aluno (recomendamos C++ (Pthread), Java ou Python) para simular o problema do Bar dos Filósofos.

Desenvolva uma classe **Filósofo** que tenha 3 estados nesta sequência: tranquilo, com sede e bebendo. O filósofo, ao passar de tranquilo para com sede, deve sortear o número de garrafas requeridas, podendo variar de 2 até n (número de arestas adjacentes ao vértice). As arestas representam o recurso compartilhado (garrafa) e tem acesso em exclusão mútua.

Utilize os seguintes tempos:

Tempo tranquilo: de 0 a n (zero a número de arestas adjacentes segundos, escolhido aleatoriamente).

Tempo com sede: tempo até conseguir todas as garrafas.

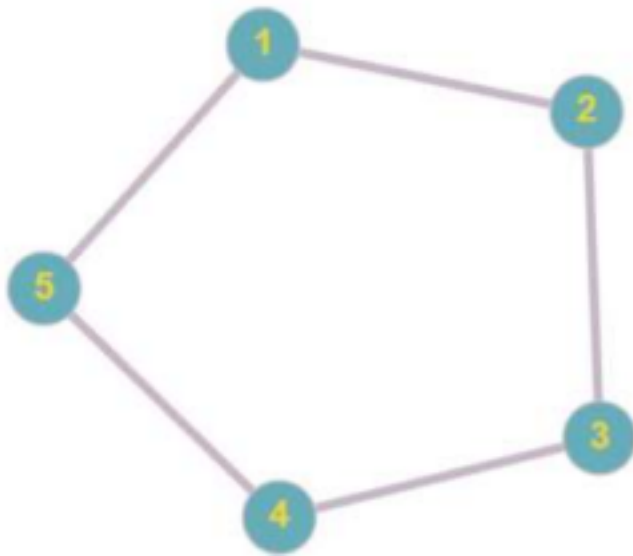
Tempo bebendo as garrafas: 1 segundo. Após beber todas as garrafas, o filósofo retorna a tranquilo.

Dado um grafo $G = (V, E)$, é possível representar G através de M , uma matriz de adjacência quadrada de ordem V , onde $M_{ij} = 1$ se o vértice i possuir uma aresta ao vértice j , ou $M_{ij} = 0$ caso contrário.

Para generalização, o programa deve ser capaz de ler um arquivo .txt onde cada linha do arquivo representa uma linha de M e cada caractere da linha assume os valores 0 ou 1, indicando as adjacências entre vértices. Considere que o índice de cada caractere nas linhas representa o n -ésimo vértice de G , ou seja, uma coluna de M . É recomendado que o programa receba como argumento o arquivo contendo a matriz de adjacências.

Utilize os seguintes grafos e respectivas matrizes de adjacência

Caso 1: Problema Jantar dos Filósofos Clássico



0, 1, 0, 0, 1

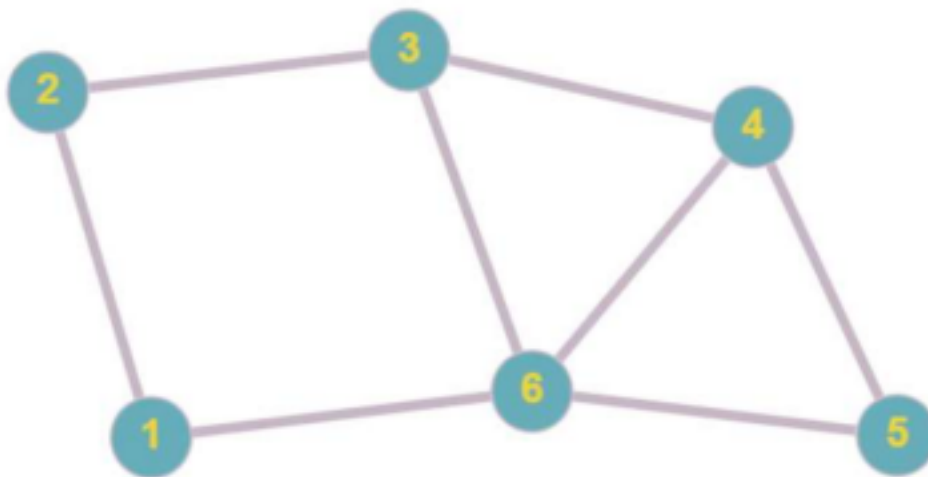
1, 0, 1, 0, 0

0, 1, 0, 1, 0

0, 0, 1, 0, 1

1, 0, 0, 1, 0

Caso 2: Problema Bar dos filósofos com 6 nós e baixa conectividade (máx 4 arestas por nó)



0, 1, 0, 0, 0, 1

1, 0, 1, 0, 0, 0

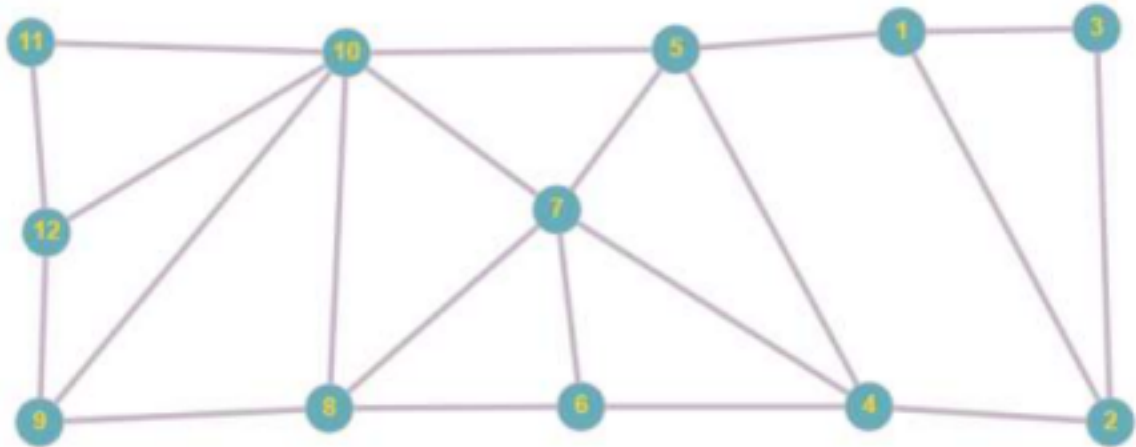
0, 1, 0, 1, 0, 1

0, 0, 1, 0, 1, 1

0, 0, 0, 1, 0, 1

1, 0, 1, 1, 1, 0

Caso 3: Problema Bar dos filósofos com 12 nós e alta conectividade (máx 6 arestas por nó)



0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0
1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0
1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
0, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0
1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0
0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0
0, 0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0
0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0
0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1
0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1
0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1
0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0

Cada filósofo deve beber 6 vezes para os exemplos 1 e 2 e deve beber 3 vezes para o exemplo 3 (uma simulação deve demorar pouco mais de 2 minutos).

Ao final da execução mostrar os tempos de cada filósofo levou em cada estado (tranquilo, com sede e bebendo).

Tempo total para execução e tempo de espera (com sede) médio de cada filósofo para avaliar a starvation.

Será considerado correto o programa que não apresentar deadlock durante a execução e os tempos médios de espera dos filósofos com mesmo número de arestas seja equilibrada (não houve starvation).

Solução do Problema

Existem várias soluções para o problema, que tem sido apresentada na literatura nos últimos anos. Para o trabalho deverão ser implementados, pelo menos, duas soluções que deverão ser comparadas pelo tempo de espera de cada filósofo e o tempo total do ciclo, sem ocorrência de deadlock.

- 1) Numeração dos Garrafas (Ordenação de Recursos). Cada aresta recebe um número e o filósofo sempre pega a garrafa de menor número primeiro e demais em sequência.
- 2) Solução do Árbitro (Garçom). Um árbitro controla quem pode pegar as garrafas. O filósofo pede permissão ao garçom antes de pegar qualquer garrafa. O garçom garante no máximo $N-1$ filósofos tentando beber ao mesmo tempo.
- 3) Chandy-Misra (Garrafas com Tokens distribuído). Cada garrafa tem um dos estados: cheia ou vazia. Inicialmente todas as garrafas estão vazias. Quando o filósofo fica com sede, envia pedidos para todos os vizinhos solicitando o recurso. Ao receber a mensagem, se estiver bebendo, aguarda terminar. Se o filósofo não estiver bebendo e a garrafa estiver vazia, enche a garrafa e entrega ao solicitante. Quando o filósofo termina de beber, coloca a garrafa em vazia. Se um filósofo receber mais de um pedido, serve na ordem de chegada.
- 4) Aleatoriedade (Randomized Backoff). Se o filósofo não consegue todas as garrafas, ele solta as que pegou e espera um tempo aleatório antes de tentar novamente.

Dica

Sugerimos dar uma leitura em alguns artigos que propõe soluções para esse problema: The Drinking Philosophers Problem de Chandy e Misra

[HTTPS://DL.ACM.ORG/DOI/PDF/10.1145/1780.1804](https://dl.acm.org/doi/pdf/10.1145/1780.1804)

On Distributed Resource Handling:Dining, Drinking and Mobile Philosophers de Filou Tsigas

[HTTPS://CITSEERX.IST.PSU.EDU/DOCUMENT?REPID=REP1&TYPE=PDF&DOI=1A7E730D265A96A3E795AA0CE277F697676B0746](https://ciseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=1A7E730D265A96A3E795AA0CE277F697676B0746)

An Efficient Solution to the Drinking Philosophers Problem and its Extensions de Ginat, Shankar e Agrawala

[HTTPS://LINK.SPRINGER.COM/CHAPTER/10.1007/3-540-51687-5_34](https://link.springer.com/chapter/10.1007/3-540-51687-5_34)